

Tema 3. Códigos de fuente binarios sin prefijo

3.1 Introducción

El objetivo principal de la compresión de datos es eliminar la redundancia presente en los datos para minimizar el espacio de almacenamiento o el tiempo de transmisión.

Ejemplo 3.1. Un lanzamiento de dado justo se puede modelar como una variable aleatoria con el alfabeto $\{1, 2, 3, 4, 5, 6\}$ y la distribución de probabilidad $\{\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\}$. Podemos utilizar la representación binaria directa y obtener

$$1 \mapsto 001 \quad 2 \mapsto 010 \quad 3 \mapsto 011 \quad 4 \mapsto 100 \quad 5 \mapsto 101 \quad 6 \mapsto 110. \quad (3.1)$$

Hay alternativas a esto, sin embargo. Por ejemplo,

$$1 \mapsto 00 \quad 2 \mapsto 01 \quad 3 \mapsto 100 \quad 4 \mapsto 101 \quad 5 \mapsto 110 \quad 6 \mapsto 111 \quad (3.2)$$

o incluso

$$1 \mapsto 1100 \quad 2 \mapsto 1010 \quad 3 \mapsto 0110 \quad 4 \mapsto 1001 \quad 5 \mapsto 0101 \quad 6 \mapsto 0011. \quad (3.3)$$

Los anteriores son mapas uno a uno (biyectivos); no tienen pérdidas, es decir, que al observar la cadena binaria, el mensaje original siempre se puede recuperar. Por ejemplo, si el resultado de varios lanzamientos de dado se codifica utilizando el código fuente en (3.2) como 1000100, sabemos que corresponde a la secuencia fuente (3, 2, 1). La misma secuencia codificada con el código fuente en (3.3) sería 011010101100, mucho más larga.

En general, mediremos la eficiencia de un código utilizando la cantidad esperada de bits necesarios para representar una fuente. Para una fuente que toma valores en $\{1, \dots, r\}$ con distribución de probabilidad $\{p_1, \dots, p_r\}$, la **longitud media** del código viene dada por

$$L_{av} = \sum_{j=1}^r p_j \ell_j \quad (3.4)$$

donde ℓ_j es la longitud (medida en dígitos binarios) de la cadena binaria asociada al símbolo j .

Ejemplo 3.2. Según el código fuente (3.1) del ejemplo 3.1, todos los símbolos de la fuente se asignan a cadenas binarias de longitud 3. Por tanto, $\ell_j = 3$ para todos los $j \in \{1, \dots, 6\}$ del alfabeto y la longitud media es simplemente $L_{av} = 3$ bits por símbolo. De la misma manera, $L_{av} = 4$ bits por símbolo para el código de (3.3). El código de (3.2) tiene longitudes variables: $\ell_j = 2$ para $j \in \{1, 2\}$ y $\ell_j = 3$ para $j \in \{3, 4, 5, 6\}$. Por tanto, $L_{av} = \frac{8}{3} \approx 2,667$ bits por símbolo. El código (3.2) proporciona claramente una manera más eficiente (más corta) de representar la fuente.

Ejemplo 3.3. Te gustaría configurar tu propio sistema telefónico que te conecte con tus 3 mejores amigos mediante códigos binarios. El número de teléfono será una **palabra código** y el conjunto de palabras código es el **código o diccionario**. Sabes que llamarás a Bernat un 50 % de las veces, y que llamarás a Alba y Carola un 25 % de las veces cada una. La fuente se puede modelar mediante el alfabeto $\{Alba, Bernat, Carola\}$ con distribución de probabilidad $\{\frac{1}{4}, \frac{1}{2}, \frac{1}{4}\}$. Ahora examinemos varios métodos para producir los números de teléfono indicados en la tabla 1 siguiente. En el código (a), Alba y Bernat tienen el mismo número, por tanto, el sistema no se conectará correctamente. El código (b) funcionará, ya que cada uno tiene un número de teléfono diferente, pero los números son muy largos y el sistema se vuelve ineficiente. El código (c) es un código más corto, pero obsérvese que el sistema no funcionará: cuando marcamos 10 no está claro si quiere decir “Carola” o bien “Bernat, Alba”. El código (c) no es **únicamente decodificable**. El código (d) es el primer código eficiente que funciona y se puede decodificar de manera única, pero ¿por qué asignamos 00 a Alba si cuando marcamos 0 está claro que queremos llamar a Alba? Una mejora es el código (e): es únicamente decodificable, pero no es el más eficiente, ya que el código está asignando una cadena de 2 bits al símbolo más probable. Finalmente, (f) es el **código óptimo** con una longitud media de $L_{av} = 1,5$ bits por símbolo. Es interesante señalar que una vez terminamos de marcar, el sistema inmediatamente sabe que hemos terminado de marcar. Esto es porque ninguna palabra código es un prefijo de ninguna otra, es decir, el código es **sin prefijo**.

Cuadro 1: Phone numbers for Example 3.3.

Friend	Probability	(a)	(b)	(c)	(d)	(e)	(f)
Alba	0.25	0011	001101	0	00	0	10
Bernat	0.5	0011	001110	1	11	11	0
Carola	0.25	1100	110000	10	10	10	11

3.2 Códigos sin prefijo

Los códigos sin prefijo son códigos de longitud variable que garantizan una decodificabilidad única y, diseñados correctamente, son eficientes en términos de longitud media, es decir, minimizan (3.4).

Considere una fuente discreta sin memoria U que toma valores sobre el alfabeto $\{1, \dots, r\}$. Un código fuente binario es el conjunto de palabras código $\mathcal{C} = \{\mathcal{C}(1), \mathcal{C}(2), \dots, \mathcal{C}(r)\}$, donde cada $\mathcal{C}(j)$ es la secuencia binaria utilizada para representar el símbolo fuente $j \in \{1, \dots, r\}$. Por ejemplo, el código (f) de la tabla 1 para la fuente $\{\text{Alba}, \text{Bernat}, \text{Carola}\}$ viene dado por $\mathcal{C}(\text{Alba}) = 10$, $\mathcal{C}(\text{Bernat}) = 0$ y $\mathcal{C}(\text{Carola}) = 11$, o simplemente el conjunto $\{10, 0, 11\}$. Una secuencia fuente de longitud n es la secuencia $(u_1, \dots, u_n)$ donde cada $u_i \in \{1, \dots, r\}$. Una codificación símbolo a símbolo consiste en $\mathcal{C}(u_1) \cdots \mathcal{C}(u_n)$, la concatenación de las palabras código. Por ejemplo, la secuencia fuente (Alba, Alba, Carola, Bernat, Carola) de longitud $n = 5$ está codificada como 101011011, una secuencia binaria de longitud de 9 bits.

Un código se denomina **únicamente decodificable** si no existe ningún par de secuencias diferentes $(u_1, \dots, u_n) \neq (u'_1, \dots, u'_m)$ de manera que $\mathcal{C}(u_1) \cdots \mathcal{C}(u_n) = \mathcal{C}(u'_1) \cdots \mathcal{C}(u'_m)$, es decir, para que un código sea decodificable de manera única, no se pueden codificar dos secuencias fuente diferentes en la misma palabra código. Recuerde el sistema telefónico (c) del ejemplo 3.3. Un código se denomina **sin prefijo** si ninguna palabra código es un prefijo de ninguna otra palabra código.

Teorema 3.4. Los códigos sin prefijo se pueden decodificar de manera única.

Problema 3.5. Considere los códigos $\mathcal{C}_1 = \{0, 10, 110, 111\}$ y $\mathcal{C}_2 = \{0, 01, 011, 111\}$ para una fuente $\{a, b, c, d\}$. ¿Son los códigos únicamente decodificables y/o libres de prefijo? Decodifique la secuencia 0011011110 utilizando los dos códigos.

Solución: El código $\mathcal{C}_1$ es libre de prefijo porque ninguna palabra código es un prefijo de ninguna otra y, por tanto, el código se puede decodificar de manera única. El código $\mathcal{C}_2$ no es libre de prefijo porque $\mathcal{C}(a)$ es el prefijo de $\mathcal{C}(b)$ y $\mathcal{C}(c)$, mientras que $\mathcal{C}(b)$ también es el prefijo de $\mathcal{C}(c)$. Obsérvese, sin embargo, que el código es únicamente decodificable porque el bit 0 es como una coma cuando se lee de derecha a izquierda. Utilizando el primer código, la secuencia a decodificar se puede agrupar de manera única como 0, 0, 110, 111, 10. Como $\mathcal{C}^{-1}(0) = a$, $\mathcal{C}^{-1}(110) = c$, $\mathcal{C}^{-1}(111) = d$ y $\mathcal{C}^{-1}(10) = b$, decodificamos la secuencia como (a, a, c, d, b) . Para el segundo código, hay que leer de derecha a izquierda como 0, 011, 01, 111, 0 para obtener (a, c, b, d, a) .

Cualquier código binario se puede representar como un árbol binario (véase la figura 3.1). Tome cualquier palabra código y considere cada bit como una decisión sobre si subir (0) o bajar (1). Así, cada palabra código puede ser representada por un camino particular en el árbol, como los dos ejemplos de la figura 3.1. En los árboles, el nodo raíz se denota con un cuadrado, los nodos intermedios con círculos pequeños y los nodos hoja como círculos grises. Las aristas se representan con una línea sólida. Las etiquetas de la derecha denotan secuencias binarias. De manera similar, en la figura 3.2, ilustramos las representaciones en árbol binario de los códigos (c), (d), (e) y (f) del ejemplo 3.3. El nodo del círculo blanco indica una hoja sin palabra código. Obsérvese que el código (c) no es libre de prefijos y tiene una palabra código a lo largo del camino hacia otra palabra código. Así, un código es libre de prefijos si en su representación en árbol todas las palabras código aparecen como hojas en el árbol.

El siguiente es un resultado importante relacionado con la existencia de códigos sin prefijo y árboles binarios.

Teorema 3.6 (Desigualdad de Kraft). Considere un código fuente binario $\mathcal{C}$ para una fuente con alfabeto $\{1, \dots, r\}$ y distribución de probabilidad $\{p_1, \dots, p_r\}$. Entonces, si el código es libre de prefijos, satisface

$$\sum_{j=1}^r 2^{-\ell_j} \leq 1. \quad (3.5)$$

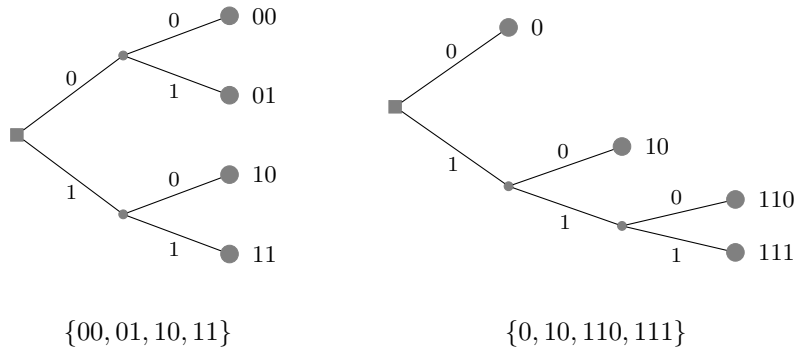


Figura 3.1: Representación en árbol de dos códigos fuente binarios.

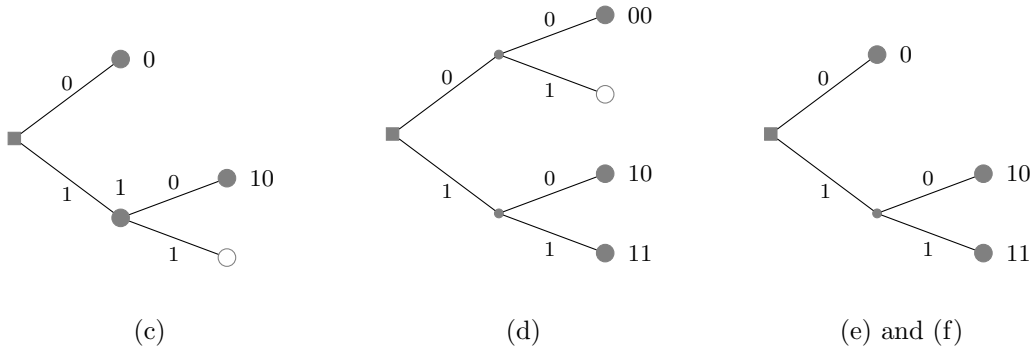


Figura 3.2: Representación en árbol de algunos códigos fuente del ejemplo 3.3.

Además, existe un código sin prefijo con longitudes ℓ_j para $j \in \{1, \dots, r\}$ que satisface (3.5).

Ejemplo 3.7. Un código con distribución de longitudes $\{\ell_1, \dots, \ell_r\} = \{3, 4, 4, 4, 4\}$ satisface la desigualdad de Kraft ya que

$$\sum_{j=1}^r 2^{-\ell_j} = 2^{-3} + 4 \cdot 2^{-4} = \frac{1}{8} + \frac{4}{16} = \frac{3}{8} \leq 1. \quad (3.6)$$

Por tanto, existe un código sin prefijo con estas longitudes, por ejemplo $\{000, 0010, 0011, 1100, 1101\}$. En cambio, para una distribución de longitudes $\{1, 2, 3, 3, 4\}$, no podemos construir un código sin prefijos porque

$$2^{-1} + 2^{-2} + 2 \cdot 2^{-3} + 2^{-4} = \frac{17}{16} \geq 1 \quad (3.7)$$

Un posible intento de construir este código se muestra en la figura 3.3, donde hay una palabra código de longitud 4 que no se puede conectar en ningún sitio. No es posible construir este código.

Terminamos esta lección afirmando un resultado fundamental en la compresión de datos.

Teorema 3.8. La longitud media de cualquier código de longitudes sin prefijo $\{\ell_1, \dots, \ell_r\}$ utilizado para representar una fuente U que toma valores en $\{1, \dots, r\}$ con distribución de probabilidad $\{p_1, \dots, p_r\}$ satisface

$$L_{\text{av}} \geq H(U), \quad (3.8)$$

donde $H(U)$ es **la entropía de la fuente**, medida en bits, que viene dada por

$$H(U) = - \sum_{j=1}^r p_j \log_2 p_j. \quad (3.9)$$

Nótese que para cumplir la desigualdad (3.8) con igualdad, las longitudes deberían ser tales que $2^{-\ell_j} = p_j$ exactamente para todo $j \in \{1, \dots, r\}$.

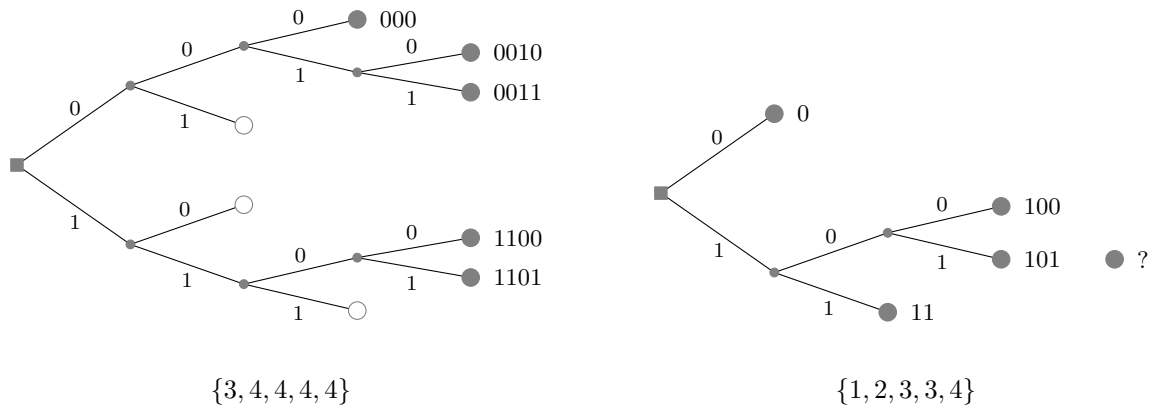


Figura 3.3: Ejemplo de representación en árbol de los códigos fuente del ejemplo 3.7.

Problema 3.9. Considere una fuente U que toma los valores $\{1, 2, 3\}$ con la misma probabilidad. Encuentre un código sin prefijo con distribución de longitud $\{1, 2, 2\}$ y compruebe que se cumplen los teoremas 3.6 y 3.8.

Solución: Un posible código sin prefijo con estas longitudes viene dado por $\{0, 10, 11\}$. El código es únicamente decodificable por el Teorema 3.4, satisface la desigualdad de Kraft en el Teorema 3.6 con igualdad puesto que $\sum_{j=1}^r 2^{-\ell_j} = 1$. Esto significa que el árbol binario asociado a $\{0, 10, 11\}$ no tiene hojas vacías. Cuando se utiliza para representar la fuente U , tiene una longitud media de $L_{av} = \sum_{j=1}^r p_j \ell_j = \frac{5}{3} \approx 1,667$ bits por símbolo, y está acotado por la entropía de la fuente desde abajo ya que

$$H(U) = - \sum_{j=1}^r p_j \log_2 p_j = \log_2 3 \approx 1,585. \quad (3.10)$$

El resultado de (3.8) indica que la entropía $H(U)$ es la longitud media mínima necesaria para representar una fuente U , no podemos representarla sin errores con menos bits por símbolo. De todos los posibles códigos sin prefijo, queremos encontrar el que minimice la longitud media en (3.4). Este será el tema de nuestra próxima lección.